

EPC-INV010 - Audit & Observability

Referrals and Appointments

6 August, 2026

Table of Contents

1	Introduction	4
1.1	Audit layers.....	4
2	Glossary.....	5
3	Requirements	6
3.1	Requirement 1: Audit Record Creation on Write Operations.....	6
3.1.1	Acceptance Criteria	6
3.2	Requirement 2: Actor Identity from Bearer Token	6
3.2.1	Acceptance Criteria	7
3.3	Requirement 3: Audit Coverage for All Resource Types	7
3.3.1	Acceptance Criteria	7
3.4	Requirement 4: Audit Query API – Search and Retrieval	8
3.4.1	Acceptance Criteria	8
3.5	Requirement 5: Audit Record Content	9
3.5.1	Acceptance Criteria	9
3.6	Requirement 6: Access Control for Audit Query API	11
3.6.1	Acceptance Criteria	11
3.7	Requirement 7: Audit Record Immutability and Retention	11
3.7.1	Acceptance Criteria	11
3.8	Requirement 8: Audit Query API – Error Handling.....	12
3.8.1	Acceptance Criteria	12
3.9	Requirement 9: API Gateway Audit (Apigee / Splunk).....	12
3.9.1	Acceptance Criteria	12
3.10	Requirement 10: Structured Logging.....	13
3.10.1	Acceptance Criteria	13
3.11	Requirement 11: Distributed Tracing.....	14
3.11.1	Acceptance Criteria	14
3.12	Requirement 12: Health Check Endpoint.....	14
3.12.1	Acceptance Criteria	14
3.13	Requirement 13: Metrics and Instrumentation	15

3.13.1	Acceptance Criteria	15
3.14	Requirement 14: Alerting Thresholds	15
3.14.1	Acceptance Criteria	16
3.15	Requirement 15: Log Retention and Accessibility	16
3.15.1	Acceptance Criteria	16
4	Endpoint catalogue NFR.....	17
5	Suggested Design	20
5.1	Design Document – Endpoint Catalog Audit & Observability	20
5.1.1	Overview	20
5.1.2	Architecture Diagram.....	21
5.1.3	Component Design	21
5.1.4	Log Retention Strategy	32
5.1.5	Security Considerations	33
5.1.6	Cost Estimate (Indicative)	33
5.1.7	Summary of AWS Services Used	34

1 Introduction

The Endpoint Catalog Audit Trail feature provides a complete, immutable history of all changes made to resources managed by the Endpoint Catalog API. The audit trail covers all four resource types – HealthcareService, Template, Endpoint, and List – and records who made each change, from which organisation, when, and what kind of change it was. A dedicated query API allows Endpoint Administrators and support staff to search and retrieve audit records using a range of filters.

The feature satisfies the original user story EPCSe006:

As an Endpoint Administrator, I want a full audit history of any changes applied to an endpoint so that I have a clear view of when endpoints are added, amended, changed status, etc. for audit and support purposes.

The scope has been extended to cover all resource types in the Endpoint Catalog, not just Endpoints.

1.1 Audit layers

The Endpoint Catalog has two distinct audit layers that together provide a complete audit picture:

Layer	Mechanism	Scope	Owner
API gateway audit	Apigee → Splunk	All inbound HTTP requests at the internet-facing API boundary, regardless of outcome	Platform / infrastructure team
Application audit	Audit_Service (this feature)	Successful write operations on catalogued FHIR resources, with business-level context	Endpoint Catalog application

The Apigee/Splunk layer captures low-level request telemetry (e.g. caller IP, request path, HTTP method, response code, latency). The exact data items to be captured at this layer are **to be agreed by the team** – they are out of scope for this requirements document but must be defined before the service goes into production.

The application audit layer (this feature) captures business-level change events with resource context and actor identity derived from the bearer token. These two layers are complementary: the gateway layer provides a complete request log; the application layer provides a queryable, resource-centric change history.

2 Glossary

- **Audit_Service:** The component of the Endpoint Catalog API responsible for recording and retrieving audit records.
 - **Audit_Record:** An immutable record capturing a single change event on a catalogued resource.
 - **Actor:** The authenticated user who performed the change, identified by claims in the bearer token.
 - **Actor_Organisation:** The ODS code of the organisation associated with the Actor, as asserted in the bearer token claims.
 - **Resource_Type:** One of the four catalogued FHIR resource types: `HealthcareService`, `Template`, `Endpoint`, or `List`.
 - **Change_Type:** A coded value describing the nature of the change. Valid values: `created`, `updated`, `deleted`, `status-changed`.
 - **Bearer_Token:** The OAuth 2.0 access token supplied by the caller in the `Authorization` header, issued by NHS CIS2.
 - **ODS_Code:** An NHS Organisation Data Service code uniquely identifying an NHS organisation.
 - **Product_Identifier:** The identifier of a product or programme associated with an Endpoint or Template (e.g. a BaRS product code).
 - **Endpoint_Identifier:** The logical identifier of an Endpoint resource within the Endpoint Catalog.
 - **Healthcare_Service_Identifier:** The logical identifier of a HealthcareService resource within the Endpoint Catalog.
 - **Audit_Query_API:** The read-only API endpoint that allows callers to search and retrieve Audit_Records.
 - **UTC:** Coordinated Universal Time. All timestamps in the system are stored and returned in UTC.
 - **Apigee_Audit:** The audit mechanism provided by the Apigee API gateway for all inbound requests at the internet-facing API boundary.
 - **Splunk:** The current log aggregation and search platform used to store and query Apigee_Audit data.
-

3 Requirements

3.1 Requirement 1: Audit Record Creation on Write Operations

User Story: As an Endpoint Administrator, I want every write operation on any catalogued resource to be automatically recorded, so that no change can occur without leaving a traceable audit entry.

3.1.1 Acceptance Criteria

1. WHEN a POST , PUT , PATCH , or DELETE request on a HealthcareService , Template , Endpoint , or List resource succeeds, THE Audit_Service SHALL create an Audit_Record for that operation.
 2. WHEN an Audit_Record is created, THE Audit_Service SHALL record the following fields:
 - actorId : the user identifier extracted from the Bearer_Token claims
 - actorOrganisation : the ODS_Code extracted from the Bearer_Token organisation claim
 - timestamp : the date and time of the change in UTC, to millisecond precision
 - resourceType : the Resource_Type of the changed resource
 - resourceId : the FHIR logical id of the changed resource
 - changeType : the Change_Type derived from the HTTP method and operation context
 3. WHEN a write operation fails (the API returns a 4xx or 5xx response), THE Audit_Service SHALL NOT create an Audit_Record for that operation.
 4. THE Audit_Service SHALL derive the Change_Type from the operation as follows:
 - POST resulting in a new resource → created
 - PUT or PATCH on an existing resource → updated
 - DELETE on an existing resource → deleted
 - PATCH or PUT that changes only the status field of an Endpoint or Template → status-changed
 5. THE Audit_Service SHALL store Audit_Records in a durable, append-only store such that existing records cannot be modified or deleted through any API operation.
-

3.2 Requirement 2: Actor Identity from Bearer Token

User Story: As an Endpoint Administrator, I want the audit record to capture who made each change using their authenticated identity, so that I can hold individuals and organisations accountable.

3.2.1 Acceptance Criteria

1. WHEN creating an Audit_Record, THE Audit_Service SHALL extract the Actor identity exclusively from the Bearer-Token claims and SHALL NOT use the NHSD-End-User-Organisation-ODS request header as the source of the Actor_Organisation.
 2. WHEN creating an Audit_Record, THE Audit_Service SHALL record the Actor_Organisation as the ODS_Code asserted in the Bearer-Token's organisation claim (the same claim used for ODS spoofing prevention as defined in the authorisation model).
 3. IF the Bearer-Token does not contain a resolvable user identifier claim, THEN THE Audit_Service SHALL reject the write operation with a 401 Unauthorized response and SHALL NOT create an Audit_Record.
 4. THE Audit_Service SHALL store the actorId and actorOrganisation as separate, independently queryable fields on every Audit_Record.
-

3.3 Requirement 3: Audit Coverage for All Resource Types

User Story: As an Endpoint Administrator, I want the audit trail to cover all resource types in the Endpoint Catalog, so that I have a complete picture of all changes regardless of which resource was affected.

3.3.1 Acceptance Criteria

1. THE Audit_Service SHALL record Audit_Records for write operations on all four Resource_Types: HealthcareService, Template, Endpoint, and List.
 2. WHEN a POST /HealthcareService operation automatically creates an associated List resource, THE Audit_Service SHALL create two Audit_Records: one for the HealthcareService creation and one for the List creation, each with changeType: created.
 3. WHEN a PUT or PATCH on a HealthcareService automatically synchronises the associated List, THE Audit_Service SHALL create an Audit_Record for the List update with changeType: updated, in addition to the Audit_Record for the HealthcareService change.
 4. THE Audit_Service SHALL record the resourceType field on every Audit_Record using the exact FHIR resource type name: HealthcareService, Endpoint, or List. Template resources SHALL be recorded with resourceType: Template.
-

3.4 Requirement 4: Audit Query API – Search and Retrieval

User Story: As an Endpoint Administrator, I want to search the audit trail using a range of filters, so that I can quickly locate the history of a specific resource, organisation, or time period.

3.4.1 Acceptance Criteria

1. THE Audit_Query_API SHALL expose a GET /AuditEvent endpoint that returns a FHIR Bundle of type searchset containing matching Audit_Records.
2. THE Audit_Query_API SHALL support the following search parameters, each independently optional:

Parameter	Filters on
date	Exact calendar date of the change (UTC)
date-from	Start of a date range (inclusive, UTC)
date-to	End of a date range (inclusive, UTC)
organisation	actorOrganisation (ODS_Code)
product-id	Product_Identifier associated with the changed resource
endpoint-id	Endpoint_Identifier of the changed Endpoint
endpoint-status	The status value recorded at the time of the change
healthcare-service-id	Healthcare_Service_Identifier of the changed or related HealthcareService
url	The address field of the Endpoint or Template at the time of the change
connection-type	The connectionType code of the Endpoint or Template
payload-type	The payloadType code of the Endpoint or Template

3. WHEN multiple search parameters are supplied in a single request, THE Audit_Query_API SHALL apply them as a logical AND, returning only Audit_Records that satisfy all supplied parameters.
 4. WHEN `date-from` and `date-to` are both supplied, THE Audit_Query_API SHALL return Audit_Records whose `timestamp` falls within the inclusive range `[date-from 00:00:00 UTC, date-to 23:59:59 UTC]`.
 5. WHEN `date` is supplied without `date-from` or `date-to`, THE Audit_Query_API SHALL return Audit_Records whose `timestamp` falls within the calendar day `[date 00:00:00 UTC, date 23:59:59 UTC]`.
 6. WHEN no search parameters are supplied, THE Audit_Query_API SHALL return the most recent 100 Audit_Records ordered by `timestamp` descending.
 7. THE Audit_Query_API SHALL return results ordered by `timestamp` descending (most recent first) in all cases.
 8. THE Audit_Query_API SHALL support pagination using `_count` (page size) and `_offset` (zero-based record offset) parameters, and SHALL include `next` and `previous` pagination links in the Bundle `link` array where applicable.
-

3.5 Requirement 5: Audit Record Content

User Story: As an Endpoint Administrator, I want each audit record to contain enough detail to understand exactly what changed and in what context, so that I can reconstruct the history of a resource without needing to cross-reference other systems.

3.5.1 Acceptance Criteria

1. THE Audit_Service SHALL include the following fields on every Audit_Record:

Field	Type	Description
<code>id</code>	UUID	Unique identifier for the Audit_Record
<code>timestamp</code>	ISO 8601 datetime (UTC)	Date and time of the change
<code>actorId</code>	string	User identifier from the Bearer-Token
<code>actorOrganisation</code>	string	ODS_Code from the Bearer-Token organisation claim

Field	Type	Description
resourceType	string	Resource_Type of the changed resource
resourceId	string	FHIR logical id of the changed resource
changeType	string	Change_Type code

2. WHEN the changed resource is an Endpoint or Template , THE Audit_Service SHALL also record the following additional fields on the Audit_Record where they are present on the resource at the time of the change:

Field	Description
endpointStatus	The status value of the Endpoint or Template after the change
endpointUrl	The address field of the Endpoint or Template
connectionType	The connectionType code
payloadType	The payloadType code(s)
productId	The Product_Identifier, if present

3. WHEN the changed resource is an Endpoint , THE Audit_Service SHALL also record the healthcareServiceId of the HealthcareService that references the Endpoint at the time of the change.
4. WHEN the changed resource is a List , THE Audit_Service SHALL also record the healthcareServiceId of the HealthcareService referenced by List.subject .
5. THE Audit_Service SHALL record the state of the resource **after** the change for created and updated events, and the state **before** the change for deleted events.
-

3.6 Requirement 6: Access Control for Audit Query API

User Story: As an Endpoint Administrator, I want the audit query API to be protected so that only authenticated callers can retrieve audit records, and organisations can only see audit records relevant to them unless they hold an elevated role.

3.6.1 Acceptance Criteria

1. THE Audit_Query_API SHALL require a valid Bearer-Token on all requests and SHALL return `401 Unauthorized` for requests with a missing, expired, or invalid token.
 2. WHEN a caller supplies a valid Bearer-Token, THE Audit_Query_API SHALL permit the caller to retrieve Audit_Records for any resource without restriction on `actorOrganisation` (audit records are not restricted by ownership – any authenticated caller may query the full audit trail).
 3. IF a request to the Audit_Query_API is made without a valid Bearer-Token, THEN THE Audit_Query_API SHALL return a FHIR `OperationOutcome` with HTTP status `401` and error code `SEND_UNAUTHORIZED`.
 4. IF a request to the Audit_Query_API supplies an invalid search parameter name, THEN THE Audit_Query_API SHALL return a FHIR `OperationOutcome` with HTTP status `400` and error code `INVALID_PARAMETER`.
-

3.7 Requirement 7: Audit Record Immutability and Retention

User Story: As an Endpoint Administrator, I want audit records to be permanent and tamper-proof, so that the audit trail can be relied upon for compliance and support investigations.

3.7.1 Acceptance Criteria

1. THE Audit_Service SHALL NOT expose any API operation that allows an Audit_Record to be modified after creation.
 2. THE Audit_Service SHALL NOT expose any API operation that allows an Audit_Record to be deleted.
 3. THE Audit_Service SHALL retain Audit_Records for a minimum of 7 years from the date of creation.
 4. WHILE an Audit_Record exists in the store, THE Audit_Service SHALL return it in query results that match its fields, regardless of the age of the record.
-

3.8 Requirement 8: Audit Query API – Error Handling

User Story: As an Endpoint Administrator, I want the audit query API to return clear, actionable error responses when I supply invalid parameters, so that I can correct my query without needing to contact support.

3.8.1 Acceptance Criteria

1. IF `date-from` is supplied with a value that is not a valid ISO 8601 date, THEN THE `Audit_Query_API` SHALL return a FHIR `OperationOutcome` with HTTP status `400` and a `diagnostics` message identifying the invalid parameter and its value.
 2. IF `date-to` is supplied with a value that is not a valid ISO 8601 date, THEN THE `Audit_Query_API` SHALL return a FHIR `OperationOutcome` with HTTP status `400` and a `diagnostics` message identifying the invalid parameter and its value.
 3. IF `date-from` is supplied with a value that is later than `date-to`, THEN THE `Audit_Query_API` SHALL return a FHIR `OperationOutcome` with HTTP status `400` and a `diagnostics` message stating that `date-from` must not be later than `date-to`.
 4. IF `date` is supplied together with either `date-from` or `date-to`, THEN THE `Audit_Query_API` SHALL return a FHIR `OperationOutcome` with HTTP status `400` and a `diagnostics` message stating that `date` cannot be combined with `date-from` or `date-to`.
 5. WHEN the `Audit_Query_API` returns an error response, THE `Audit_Query_API` SHALL use the FHIR `OperationOutcome` resource format consistent with the error response format used by the rest of the Endpoint Catalog API.
-

3.9 Requirement 9: API Gateway Audit (Apigee / Splunk)

User Story: As a platform operator, I want all inbound requests to the Endpoint Catalog API to be logged at the gateway layer, so that I have a complete low-level request trail that complements the application-level audit records.

3.9.1 Acceptance Criteria

1. THE Apigee API gateway SHALL emit an audit log entry to Splunk for every inbound HTTP request to the Endpoint Catalog API, regardless of whether the request succeeds or fails.
2. THE team SHALL agree and document the specific data items to be captured in the `Apigee_Audit` log before the service enters production. **This is an open action – the data items are not yet defined.**
3. AS A MINIMUM, the agreed data items SHOULD include:

- Request timestamp (UTC)
 - HTTP method
 - Request path
 - HTTP response status code
 - Caller IP address or network identifier
 - `X-Request-Id` and `X-Correlation-Id` header values (to allow correlation with application logs)
 - `NHSD-End-User-Organisation-ODS` header value
4. THE Apigee_Audit log SHALL be queryable in Splunk by the platform team for support and security investigations.
 5. THE Apigee_Audit log and the Audit_Service application audit (Requirements 1–8) SHALL be treated as complementary layers. The Apigee_Audit log is NOT a substitute for the application audit, and the application audit is NOT a substitute for the Apigee_Audit log.

⚠ Open action: The full list of data items to be captured by Apigee and forwarded to Splunk must be agreed by the team. A working session should be scheduled to define these before the production readiness review.

3.10 Requirement 10: Structured Logging

User Story: As a platform operator, I want all application log output to be structured and machine-parseable, so that I can search, filter, and alert on log data without relying on fragile text parsing.

3.10.1 Acceptance Criteria

1. THE Endpoint Catalog API SHALL emit all application logs in structured JSON format.
2. EVERY log entry SHALL include at minimum:
 - `timestamp` (ISO 8601, UTC, millisecond precision)
 - `level` (one of: `DEBUG`, `INFO`, `WARN`, `ERROR`)
 - `correlationId` (the `X-Correlation-Id` from the inbound request, if present)
 - `requestId` (the `X-Request-Id` from the inbound request, if present)
 - `message` (human-readable description of the event)
3. WHEN a log entry relates to a specific resource operation, THE log entry SHALL also include:
 - `resourceType`
 - `resourceId`

- `httpMethod`
 - `httpPath`
 - `httpStatusCode`
4. THE application SHALL NOT log sensitive data including bearer token values, patient identifiable information, or file contents. ODS codes and user identifiers from token claims MAY be logged.
-

3.11 Requirement 11: Distributed Tracing

User Story: As a platform operator, I want every request to carry a trace context through all internal components, so that I can follow a single request end-to-end when diagnosing latency or failures.

3.11.1 Acceptance Criteria

1. THE Endpoint Catalog API SHALL propagate the `X-Correlation-Id` header value through all internal service calls, database operations, and downstream requests.
 2. IF an inbound request does not include an `X-Correlation-Id` header, THE API SHALL generate a UUID and use it as the correlation ID for the duration of that request.
 3. THE API SHALL include the `X-Correlation-Id` and `X-Request-Id` in all log entries, audit records, and error responses associated with that request.
 4. WHEN the API makes calls to downstream services (e.g. object storage for attachments), THE API SHALL forward the `X-Correlation-Id` as a request header to enable end-to-end trace correlation.
-

3.12 Requirement 12: Health Check Endpoint

User Story: As a platform operator, I want a dedicated health check endpoint, so that load balancers and monitoring systems can determine whether the service is healthy without invoking business logic.

3.12.1 Acceptance Criteria

1. THE Endpoint Catalog API SHALL expose a `GET /_health` endpoint that does not require authentication.
2. WHEN all critical dependencies (database, audit store) are reachable and responsive, THE health endpoint SHALL return HTTP `200 OK` with a JSON body containing `{"status": "healthy"}`.
3. WHEN one or more critical dependencies are unreachable or unresponsive, THE health endpoint SHALL return HTTP `503 Service Unavailable` with a JSON body

containing `{"status": "unhealthy", "checks": {...}}` where `checks` lists each dependency and its individual status.

4. THE health endpoint SHALL respond within 5 seconds under all conditions. If a dependency check exceeds this timeout, it SHALL be reported as `unhealthy` rather than blocking the response.
-

3.13 Requirement 13: Metrics and Instrumentation

User Story: As a platform operator, I want the API to expose operational metrics, so that I can build dashboards and configure alerts for throughput, latency, and error rates.

3.13.1 Acceptance Criteria

1. THE Endpoint Catalog API SHALL record the following metrics for every inbound request:
 - Request count (labelled by HTTP method, path pattern, and response status code)
 - Request duration / latency (histogram, labelled by HTTP method and path pattern)
 - In-flight request count (gauge)
 2. THE Endpoint Catalog API SHALL record the following metrics for the audit store:
 - Audit record write count (counter)
 - Audit record write latency (histogram)
 - Audit record write failure count (counter)
 3. THE Endpoint Catalog API SHALL record the following metrics for the primary data store:
 - Database query count (labelled by operation type: read, write)
 - Database query latency (histogram)
 - Database connection pool utilisation (gauge: active connections / max connections)
 4. ALL metrics SHALL be exposed in a format compatible with the platform's monitoring stack (e.g. Prometheus exposition format or CloudWatch embedded metrics format – to be confirmed by the team).
 5. Metric labels SHALL NOT include high-cardinality values such as resource IDs, user IDs, or full request paths with path parameters expanded.
-

3.14 Requirement 14: Alerting Thresholds

User Story: As a platform operator, I want predefined alerting rules for key operational indicators, so that the team is notified of degradation before it impacts users.

3.14.1 Acceptance Criteria

1. THE platform team SHALL define and configure alerts for at minimum the following conditions:
 - Error rate (5xx responses) exceeds 1% of total requests over a 5-minute window
 - P95 request latency exceeds 2 seconds over a 5-minute window
 - Health check endpoint returns 503 for more than 2 consecutive checks
 - Audit record write failure rate exceeds 0 over a 1-minute window (any failure is alertable)
 - Database connection pool utilisation exceeds 80% for more than 5 minutes
 2. ALERTS SHALL be routed to the team's agreed notification channel (e.g. Slack, PagerDuty – to be confirmed).
 3. EACH alert SHALL include the X-Correlation-Id of at least one affected request (where applicable) to aid rapid investigation.
 4. THE specific threshold values in criterion 1 are initial recommendations. THE team SHALL review and adjust these based on baseline performance data gathered during integration testing.
-

3.15 Requirement 15: Log Retention and Accessibility

User Story: As a platform operator, I want application logs to be retained for a defined period and accessible through a central search tool, so that I can investigate incidents after the fact.

3.15.1 Acceptance Criteria

1. THE platform SHALL retain structured application logs for a minimum of 90 days in a searchable log aggregation system.
2. THE platform SHALL retain application logs in cold storage for a minimum of 1 year for compliance and post-incident review purposes.
3. THE log aggregation system SHALL support searching by correlationId , requestId , resourceType , resourceId , level , and time range.
4. ACCESS to application logs SHALL be restricted to authorised platform and support personnel. Logs SHALL NOT be accessible to API consumers.
5. THE specific retention periods in criteria 1 and 2 are initial recommendations. THE team SHALL confirm these align with the organisation's data retention policy.

4 Endpoint catalogue NFR

NFR-1 - Every change is recorded

As a member of the endpoint maintenance team, I want every change to a service, endpoint, supplier or template recorded automatically, so that when something breaks I can always trace how it got that way.

- Any successful create, edit, rename, endpoint switch, status change, or supplier add/remove produces an audit entry - no action changes data without leaving a record.
- If a change fails, no misleading entry is written.
- Recording is automatic - I never have to remember to log anything.

NFR-2 - Each entry says who, what, when

As a team member investigating an issue, I want each entry to show who made the change, from which organisation, and exactly when, so that I can hold the right person/org accountable and ask the right questions.

- Entry shows: user that made the change, org/ODS code, timestamp, the service/endpoint/supplier affected, and the change type (created / updated / renamed / status-changed / deleted).
- Timestamps are also shown in readable local time, not just raw UTC.

NFR-3 - Before-and-after values

As a team member fixing a bad change, I want to see what a value was before and after, so that I can understand or reverse it without guessing.

- For an edit, the entry shows old → new for the fields that actually changed (e.g. endpoint URL, status, service name, supplier).
- For a delete, it shows the last state before deletion.

NFR-4 - I can search history the same way I search everything else

As a team member, I want to search change history by ODS code, service name, endpoint and date.

- History is searchable by ODS code, service name (including partial matches), endpoint id/URL, status, org, and date range.

NFR-5-The history is trustworthy

As a team member relying on the trail to diagnose problems, I want it to be complete and tamper-proof, so that I can trust what it tells me.

- No way in the tool to alter or delete a past entry.
- The same change always produces exactly one entry -no duplicates, no gaps.
- Entries stay retrievable for 1 year.

NFR-6 - Drill-down is quick

As a team member, I want to get from a service to its endpoints, suppliers and history in a click or two, so that I'm not constantly re-searching.

- From a found service I can reach its endpoints, suppliers and change history without starting a new search.

NFR-7 - Errors are clear and actionable

As a team member, I want failures to tell me what went wrong and give me a reference I can quote, so that I can fix it myself or hand support something useful.

- Failures show a plain-language message and the specific field/value at fault.
- Every error carries a copyable reference/correlation id that ties back to the logs.
- I'm never left with a blank screen, raw stack trace or unhelpful error message.

NFR-8 - Stays responsive as it replaces other tools

As a team member, I want performance to hold as the tool takes on more services and users, so that expanding scope doesn't make my day-to-day slower.

- The current tools search feature doesn't always respond to searches.
- Planned scope expansion doesn't degrade lookup times.
- Any scheduled downtime is communicated in advance and kept outside working hours.

NFR-9 - My team and I see current, consistent data

As a team member working alongside colleagues, I want a change to show up promptly for everyone, so that two of us don't act on stale information.

- A change one person makes is visible to others within a few seconds.
- History reflects the true order in which changes happened.

5 Suggested Design

5.1 Design Document – Endpoint Catalog Audit & Observability

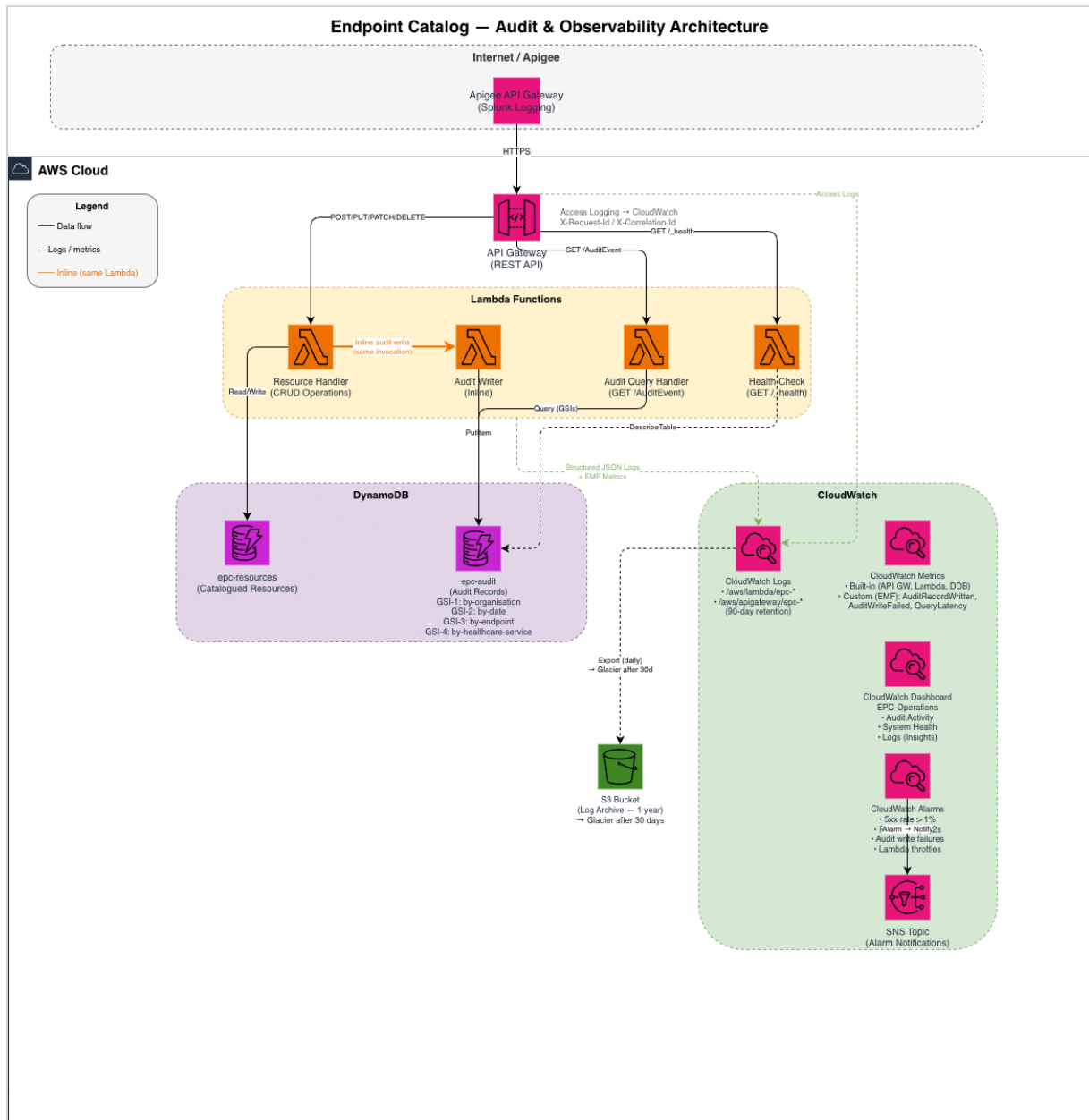
5.1.1 Overview

This design describes how the audit trail, observability, and operational dashboard requirements (Requirements 1–15) are implemented using AWS-native services. The stack is:

- **API Gateway** – internet-facing REST API (fronted by Apigee in production)
- **AWS Lambda** – compute for all API operations
- **DynamoDB** – primary data store for catalogued resources AND audit records
- **CloudWatch** – logs, metrics, alarms, and dashboards

No additional infrastructure is introduced beyond what the Endpoint Catalog already uses. The design favours out-of-the-box AWS capabilities over custom tooling.

5.1.2 Architecture Diagram



5.1.3 Component Design

5.1.3.1 1. Audit Record Storage — DynamoDB `epc-audit` Table

A dedicated DynamoDB table stores all audit records. This keeps audit data separate from operational resource data, simplifies retention policies, and avoids hot-key contention on the main table.

5.1.3.1.1 Table Schema

Attribute	Type	Description
PK	String	{resourceType}#{resourceId} – enables efficient per-resource history queries
SK	String	{timestamp}#{auditId} – ISO 8601 timestamp + UUID ensures uniqueness and natural sort order
auditId	String	UUID – unique identifier for the audit record
timestamp	String	ISO 8601 UTC datetime with millisecond precision
changeDate	String	Calendar date portion (YYYY-MM-DD) – used for date-based GSI
actorId	String	User identifier from bearer token
actorOrganisation	String	ODS code from bearer token
resourceType	String	HealthcareService, Template, Endpoint, or List
resourceId	String	FHIR logical ID of the changed resource
changeType	String	created, updated, deleted, status-changed
endpointStatus	String	(optional) Status value after the change
endpointUrl	String	(optional) Address field

Attribute	Type	Description
connectionType	String	(optional) Connection type code
payloadType	String	(optional) Payload type code(s)
productId	String	(optional) Product identifier
healthcareServiceId	String	(optional) Related HealthcareService ID
changes	List of Maps	(optional, updated / status-changed only) Field-level diff array. Each entry: { "field": "...", "from": "...", "to": "..." }

The top-level denormalised fields

(endpointStatus , endpointUrl , connectionType , payloadType , productId) reflect the resource state **after** the change (or before, for deletes). They exist to support efficient DynamoDB filter expressions on queries.

The changes array provides a human-readable narrative of what was modified. It is NOT used for query filtering – DynamoDB cannot efficiently filter on values inside a list of maps. It is used for support investigations and display in the audit dashboard.

5.1.3.1.2 Global Secondary Indexes (GSIs)

GSI	PK	SK	Purpose
GSI-1 by-organisation	actorOrganisation	timestamp	Search by organisation + date range
GSI-2 by-date	changeDate	timestamp	Search by exact date or date range
GSI-3 by-endpoint	endpointId (composite: Endpoint#{resourceId})	timestamp	Search by endpoint identifier

GSI	PK	SK	Purpose
GSI-4 by- healthcare- service	healthcareServiceId	timestamp	Search by healthcare service

5.1.3.1.3 Capacity and Scaling

- **Billing mode:** Pay-per-request (on-demand). Audit writes are proportional to API write traffic, which is low-to-moderate and bursty.
- **Point-in-time recovery (PITR):** Enabled – provides continuous backups and satisfies the immutability requirement (accidental deletes can be recovered).
- **Deletion protection:** Enabled on the table to prevent accidental table deletion.
- **TTL:** Not configured – records are retained indefinitely (7-year minimum per Requirement 7). A lifecycle policy can be added later to archive to S3 Glacier if storage costs become a concern.

5.1.3.2 2. Audit Record Creation – Inline Write in Lambda

Audit records are written **synchronously within the same Lambda invocation** that performs the resource write. This is the simplest approach and guarantees that:

- The audit record is written in the same execution context as the resource change
- If the audit write fails, the entire operation can be failed (transactional guarantee)
- No additional infrastructure (queues, streams) is needed

5.1.3.2.1 Write Flow

1. Lambda receives API Gateway event
2. Validate bearer token → extract actorId, actorOrganisation
3. Validate NHSD-End-User-Organisation-ODS against token (spoofing check)
4. Execute resource write on epc-resources table
5. IF resource write succeeds:
 - a. Construct audit record from token claims + resource state
 - b. PutItem to epc-audit table
 - c. IF audit write fails → log ERROR, return 500 to caller
6. Return success response

Why not async (SQS/DynamoDB Streams)? For this use case, the audit write adds ~5–10ms of latency (single DynamoDB PutItem). The simplicity of inline writes outweighs the marginal latency saving of async processing. Async introduces failure modes (dead-letter queues, replay logic) that add operational complexity without meaningful benefit at the expected traffic volume.

5.1.3.2.2 TransactWriteItems Alternative

For stronger consistency, the resource write and audit write can be combined into a single DynamoDB `TransactWriteItems` call. This guarantees both succeed or both fail atomically. The trade-off is a 2x write capacity cost and a 4MB transaction size limit. Recommended for production; optional for the PoC.

5.1.3.3 3. Audit Query – GET /AuditEvent Lambda

A dedicated Lambda function (or route within the existing Lambda) handles audit queries.

5.1.3.3.1 Query Strategy

Search parameter	DynamoDB access pattern
endpoint-id	Query GSI-3 (by-endpoint)
healthcare-service-id	Query GSI-4 (by-healthcare-service)
organisation	Query GSI-1 (by-organisation)
date (exact)	Query GSI-2 (by-date) with PK = date
date-from + date-to	Query GSI-2 with PK between dates (requires scan across partition keys – see below)
product-id, url, connection-type, payload-type, endpoint-status	Filter expression on any of the above queries
No parameters	Scan GSI-2 descending, limit 100

Date range queries across multiple days: GSI-2 uses `changeDate` as the partition key. A date range spanning N days requires N parallel queries (one per day) merged client-side. For ranges up to 90 days this is acceptable. For very large ranges, consider a scan with a filter – acceptable given the low query volume expected.

5.1.3.3.2 Pagination

- `_count` maps to DynamoDB `Limit` parameter

- `_offset` is implemented using `ExclusiveStartKey` (the last evaluated key from the previous page is encoded as an opaque base64 token in the Bundle `next` link)
 - Results are always sorted by `timestamp` descending (SK sort order)
-

5.1.3.4 4. Structured Logging – CloudWatch Logs

5.1.3.4.1 Lambda Log Configuration

- **Log format:** JSON (set via Lambda environment variable `AWS_LAMBDA_LOG_FORMAT=JSON` or configured in the Lambda runtime logging settings – available natively since Nov 2023)
- **Log level:** Configurable via environment variable (`LOG_LEVEL=INFO` for production)
- **Log group:** `/aws/lambda/epc-{function-name}`
- **Retention:** 90 days (set on the CloudWatch Log Group)

5.1.3.4.2 Log Structure

Every log entry emitted by the Lambda includes:

```
{
  "timestamp": "2026-05-14T10:30:00.123Z",
  "level": "INFO",
  "correlationId": "9562466f-c982-4bd5-bb0e-255e9f5e6689",
  "requestId": "c1ab3fba-6bae-4ba4-b257-5a87c44d4a91",
  "message": "Resource updated successfully",
  "resourceType": "Endpoint",
  "resourceId": "0cb21027-a246-43e6-9c7a-35b17163eab1",
  "httpMethod": "PUT",
  "httpPath": "/Endpoint/0cb21027-a246-43e6-9c7a-35b17163eab1",
  "httpStatusCode": 200,
  "actorOrganisation": "R778",
  "durationMs": 45
}
```

5.1.3.4.3 API Gateway Access Logging

- **Log group:** `/aws/apigateway/epc-access-logs`
 - **Format:** JSON with fields: `requestId`, `ip`, `httpMethod`, `resourcePath`, `status`, `responseLength`, `requestTime`, `integrationLatency`
 - **Retention:** 90 days
-

5.1.3.5 5. Distributed Tracing

5.1.3.5.1 Implementation

- `X-Correlation-Id` is mapped from the API Gateway request to the Lambda event via a request mapping or passthrough.
- If absent, the Lambda generates a UUID at the start of execution.
- The correlation ID is:
 - Included in every structured log entry
 - Written to the audit record
 - Returned in the response headers
 - Passed to any downstream AWS SDK calls via custom middleware

5.1.3.5.2 AWS X-Ray (Optional Enhancement)

- Enable X-Ray tracing on API Gateway and Lambda for automatic trace capture.
 - Provides latency breakdown (API GW → Lambda → DynamoDB) without custom code.
 - Traces are correlated with CloudWatch Logs via the `X-Correlation-Id` in log entries.
-

5.1.3.6 6. Health Check – `GET /_health`

5.1.3.6.1 Implementation

A lightweight Lambda handler (or route) that:

1. Attempts a DynamoDB `DescribeTable` call on both `epc-resources` and `epc-audit` tables (low-cost, confirms connectivity)
2. Returns aggregated status

5.1.3.6.2 Response

```
// Healthy
{
  "status": "healthy",
  "checks": {
    "dynamodb-resources": "ok",
    "dynamodb-audit": "ok"
  },
  "timestamp": "2026-05-14T10:30:00.123Z"
}

// Unhealthy
{
```

```

"status": "unhealthy",
"checks": {
  "dynamodb-resources": "ok",
  "dynamodb-audit": "timeout"
},
"timestamp": "2026-05-14T10:30:00.123Z"
}

```

5.1.3.6.3 API Gateway Configuration

- Route: GET /_health
- No authorizer attached (public, no bearer token required)
- Timeout: 5 seconds (Lambda timeout for this route)

5.1.3.7 7. Metrics – CloudWatch Custom Metrics + Built-in Metrics

5.1.3.7.1 Built-in Metrics (No Code Required)

Source	Metrics	Notes
API Gateway	Count, 4XXError, 5XXError, Latency, IntegrationLatency	Per-stage, per-route
Lambda	Invocations, Errors, Throttles, Duration, ConcurrentExecutions	Per-function
DynamoDB	ConsumedReadCapacityUnits, ConsumedWriteCapacityUnits, ThrottledRequests, SuccessfulRequestLatency	Per-table, per-GSI

5.1.3.7.2 Custom Metrics (Emitted via CloudWatch Embedded Metric Format)

The Lambda emits custom metrics using the [CloudWatch Embedded Metric Format \(EMF\)](#)¹. This allows metrics to be emitted as structured log lines – no separate PutMetricData API calls needed.

Metric	Type	Dimensions	Description
AuditRecordWritten	Counter	resourceType, changeType	Incremented on each successful audit write

1. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch_Embedded_Metric_Format.html

Metric	Type	Dimensions	Description
AuditRecordWriteFailed	Counter	resourceType	Incremented on audit write failure
AuditQueryExecuted	Counter	—	Incremented on each audit query
AuditQueryLatency	Timer	—	Duration of audit query execution
RequestDuration	Timer	httpMethod, routePattern	End-to-end Lambda execution time

5.1.3.7.3 EMF Log Example

```
{
  "_aws": {
    "Timestamp": 1715684400123,
    "CloudWatchMetrics": [{
      "Namespace": "EPC/Audit",
      "Dimensions": [{"resourceType", "changeType"}],
      "Metrics": [{"Name": "AuditRecordWritten", "Unit": "Count"}]
    }]
  },
  "resourceType": "Endpoint",
  "changeType": "updated",
  "AuditRecordWritten": 1
}
```

5.1.3.8 8. Alerting – CloudWatch Alarms

Alarm	Metric	Condition	Period	Action
High error rate	API GW 5XXError / Count	> 1%	5 min	SNS → team notification
High latency	API GW Latency P95	> 2000 ms	5 min	SNS → team notification

Alarm	Metric	Condition	Period	Action
Health check failing	Custom HealthCheckStatus	= 0 (unhealthy) for 2 consecutive	1 min	SNS → team notification
Audit write failure	Custom AuditRecordWriteFailed	> 0	1 min	SNS → team notification
Lambda throttles	Lambda Throttles	> 0	1 min	SNS → team notification
DynamoDB throttles	DynamoDB ThrottledRequests	> 0	5 min	SNS → team notification

Notification: All alarms publish to an SNS topic. The team subscribes via email, Slack (via AWS Chatbot), or PagerDuty (via SNS → HTTPS endpoint).

5.1.3.9 9. CloudWatch Dashboard

A single CloudWatch dashboard named EPC-Operations provides three sections:

5.1.3.9.1 Section 1: Audit Activity

Widget	Type	Data Source
Audit records created (last 24h)	Number	AuditRecordWritten sum
Audit records by resource type	Stacked bar chart	AuditRecordWritten grouped by resourceType dimension
Audit records by change type	Pie chart	AuditRecordWritten grouped by changeType dimension
Audit query volume	Line graph	AuditQueryExecuted over time
Audit query latency (P50, P95, P99)	Line graph	AuditQueryLatency percentiles

Widget	Type	Data Source
Audit write failures	Number (alarm status)	AuditRecordWriteFailed sum

5.1.3.9.2 Section 2: System Health

Widget	Type	Data Source
API request volume	Line graph	API GW Count
Error rate (4xx / 5xx)	Line graph	API GW 4XXError , 5XXError
API latency (P50, P95, P99)	Line graph	API GW Latency percentiles
Lambda duration (P50, P95)	Line graph	Lambda Duration
Lambda errors & throttles	Line graph	Lambda Errors , Throttles
Lambda concurrent executions	Line graph	Lambda ConcurrentExecutions
DynamoDB consumed capacity	Line graph	DynamoDB ConsumedReadCapacityUnits , ConsumedWriteCapacityUnits
DynamoDB throttled requests	Number (alarm status)	DynamoDB ThrottledRequests
Health check status	Alarm status widget	HealthCheckStatus alarm
Active alarms	Alarm status widget	All EPC alarms

5.1.3.9.3 Section 3: Logs

Widget	Type	Data Source
Recent errors	Log table (query)	CloudWatch Logs Insights: <code>filter level = "ERROR"</code> from Lambda log group
Recent audit write failures	Log table (query)	CloudWatch Logs Insights: <code>filter message like "Audit write failed"</code>
Request trace lookup	Log table (query)	CloudWatch Logs Insights: <code>filter correlationId = "{input}"</code> – parameterised
Slow requests (> 1s)	Log table (query)	CloudWatch Logs Insights: <code>filter durationMs > 1000</code>

5.1.3.9.4 Dashboard-as-Code

The dashboard is defined in the infrastructure-as-code (Terraform) as a `aws_cloudwatch_dashboard` resource with the JSON body specifying all widgets. This ensures the dashboard is version-controlled and reproducible across environments.

5.1.4 Log Retention Strategy

Log source	CloudWatch retention	Archive	Notes
Lambda application logs	90 days	S3 export (1 year)	Via CloudWatch Logs subscription → S3
API Gateway access logs	90 days	S3 export (1 year)	Same mechanism
Audit records (DynamoDB)	Indefinite (in-table)	—	7-year minimum per Requirement 7

For the S3 archive:

- Use CloudWatch Logs subscription filter → Kinesis Data Firehose → S3 (Glacier after 30 days)
- Or use the native CloudWatch Logs `CreateExportTask` on a daily schedule via EventBridge
 - Lambda

5.1.5 Security Considerations

Concern	Mitigation
Audit record tampering	DynamoDB table has no <code>DeleteItem</code> or <code>UpdateItem</code> IAM permissions granted to the Lambda execution role for the <code>epc-audit</code> table. Only <code>PutItem</code> and <code>Query / GetItem</code> are permitted.
Accidental table deletion	DynamoDB deletion protection enabled
Log data sensitivity	Lambda logs do not include bearer token values or patient data. ODS codes and user IDs are logged for traceability.
Dashboard access	CloudWatch dashboard access controlled via IAM policies – only platform team roles can view
Audit query access	<code>GET /AuditEvent</code> requires a valid bearer token (same auth as all other endpoints)

5.1.6 Cost Estimate (Indicative)

Component	Cost driver	Estimate at 10k writes/day
DynamoDB <code>epc-audit</code> (on-demand)	Write: \$1.25/million WRU, Read: \$0.25/million RRU	~\$0.40/month writes, reads negligible
DynamoDB storage	\$0.25/GB/month	~\$0.10/month (first year, growing)
CloudWatch Logs	\$0.50/GB ingested	~\$2–5/month
CloudWatch Metrics (custom)	\$0.30/metric/month (first 10k)	~\$1.50/month (5 custom metrics)
CloudWatch Dashboard	\$3/dashboard/month	\$3/month
CloudWatch Alarms	\$0.10/alarm/month	~\$0.60/month (6 alarms)

Component	Cost driver	Estimate at 10k writes/day
Total incremental		~\$8–11/month

5.1.7 Summary of AWS Services Used

Requirement	AWS Service	Feature
Audit storage (Req 1, 5, 7)	DynamoDB	Dedicated table, PITR, deletion protection
Audit query (Req 4, 8)	DynamoDB + Lambda	GSI queries, filter expressions
Structured logging (Req 10)	CloudWatch Logs	JSON log format, Log Insights
Distributed tracing (Req 11)	CloudWatch Logs + X-Ray (optional)	Correlation ID propagation
Health check (Req 12)	Lambda + API Gateway	Unauthenticated route, DynamoDB connectivity check
Metrics (Req 13)	CloudWatch Metrics	Built-in (API GW, Lambda, DynamoDB) + EMF custom metrics
Alerting (Req 14)	CloudWatch Alarms + SNS	Threshold-based alarms, team notification
Log retention (Req 15)	CloudWatch Logs + S3	90-day hot, 1-year cold archive
Dashboard	CloudWatch Dashboards	Single pane: audit, health, logs
Gateway audit (Req 9)	API Gateway access logging	JSON access logs to CloudWatch